



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

EXPOSICIÓN CORTE II

**Integración de servicios REST en Svelte:
Investigación bibliográfica y manual técnico**

Materia: Aplicaciones Web

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Integrantes:

- Castro Coello Mario
- Loor Mendoza Byron
- Reinoso Vélez Eduardo
- Rivas Piloso Dayver

Quevedo - Ecuador
Febrero 2026

Índice

Investigación bibliográfica	1
I Introducción	1
II Contexto arquitectónico de las aplicaciones web modernas	1
III Svelte: un paradigma diferente en el desarrollo frontend	2
III.I Modelo de componentes en Svelte	4
III.II Reactividad declarativa y actualización eficiente del DOM	4
III.III Gestión de estado en aplicaciones Svelte	4
III.IV Ciclo de vida y patrones de integración con servicios REST	5
IV Consumo de servicios web REST con Svelte	5
IV.I Gestión de estados derivados de peticiones asíncronas	6
IV.II Consumo seguro de servicios REST	7
V Operaciones CRUD, consultas y visualización de datos en Svelte	8
V.I Implementación de operaciones CRUD	8
V.II Tablas y consultas de datos	8
V.III Generación de reportes	8
VI Buenas prácticas en el consumo de servicios REST desde Svelte	9
VII Conclusiones	9
Manual técnico	12
1 Descripción del manual	12
2 Operaciones CRUD	14
3 Consultas y filtros dinámicos	23
4 Tablas HTML dinámicas y visualización de datos	37
5 Generación de reportes e impresión en PDF	42
6 Conclusiones del manual técnico	47

Investigación bibliográfica

Investigación bibliográfica: Consumo de servicios web REST desde Svelte

I. Introducción

El desarrollo frontend ha evolucionado desde la manipulación directa del DOM mediante JavaScript hacia frameworks que abstraen la construcción de interfaces y la gestión del estado. En este contexto, Svelte surge como una alternativa que traslada parte del trabajo de actualización de la interfaz al proceso de compilación, buscando reducir sobrecarga de ejecución y simplificar la construcción de interfaces reactivas.

Este documento analiza el uso de Svelte como framework frontend en escenarios donde la aplicación cliente consume servicios web REST provistos por un backend. El análisis se centra en arquitecturas distribuidas basadas en Spring Boot y en la implementación de operaciones CRUD, consultas y visualización de datos desde la capa de presentación. Se abordan los fundamentos necesarios para comprender cómo se realiza el intercambio de datos entre capas y cómo se integra el control de acceso mediante autenticación y autorización, aspectos indispensables cuando las operaciones del sistema dependen de recursos protegidos.

La investigación es relevante porque el consumo seguro y mantenible de APIs es un componente central en aplicaciones modernas: exige definir contratos claros, manejar estados asíncronos (carga, éxito y error) y aplicar controles de seguridad coherentes para evitar accesos no autorizados y preservar la integridad de los datos.

II. Contexto arquitectónico de las aplicaciones web modernas

Las aplicaciones web modernas se desarrollan predominantemente bajo arquitecturas distribuidas que establecen una separación clara entre la capa de presentación (frontend) y la capa de lógica de negocio y acceso a datos (backend). Este enfoque permite que distintos tipos de clientes, como aplicaciones web, aplicaciones móviles o sistemas de terceros, consuman los mismos recursos de información a través de interfaces estandarizadas.

En este contexto, los servicios web REST se consolidan como el mecanismo principal de comunicación entre frontend y backend. Mediante el uso de protocolos HTTP y representaciones ligeras de datos, generalmente en formato JSON, los servicios REST permiten exponer funcionalidades del dominio de la aplicación de forma desacoplada, interoperable y escalable [1, 2]. Esta aproximación evita el acceso directo a la base de datos desde la capa cliente, reduciendo riesgos de seguridad y favoreciendo una mejor gobernanza de los datos.

El backend, comúnmente implementado con frameworks como Spring Boot, actúa como intermediario entre la base de datos y los clientes, encapsulando la lógica de negocio y aplicando políticas de validación, autorización y control de acceso [3]. A través de controladores REST, el backend define endpoints que representan recursos del sistema y operaciones sobre estos, utilizando métodos HTTP estándar como GET, POST, PUT/-

PATCH y DELETE. Este modelo establece un contrato claro entre cliente y servidor, facilitando la evolución independiente de ambas capas [4].

Por su parte, el frontend asume la responsabilidad exclusiva de la interacción con el usuario y la representación de la información. Los frameworks frontend modernos proporcionan mecanismos para gestionar estados derivados de peticiones asíncronas, tales como carga en progreso, recepción de datos o manejo de errores. Esta capacidad es esencial cuando el frontend depende de servicios web externos, ya que la experiencia de usuario debe adaptarse dinámicamente a la latencia y disponibilidad de los servicios backend [5, 6].

En este escenario arquitectónico, la seguridad adquiere un rol central. El uso de mecanismos de autenticación y autorización basados en tokens, como JWT, permite que el backend valide la identidad del usuario y determine los permisos asociados a cada solicitud. De este modo, el frontend actúa como consumidor de servicios protegidos, mientras que el backend mantiene el control efectivo sobre el acceso a los recursos y la integridad de los datos [7].

La Figura 1 ilustra una arquitectura web típica basada en servicios REST, donde el frontend desarrollado en Svelte consume APIs expuestas por un backend en Spring Boot, el cual a su vez gestiona el acceso a la base de datos y los mecanismos de seguridad.

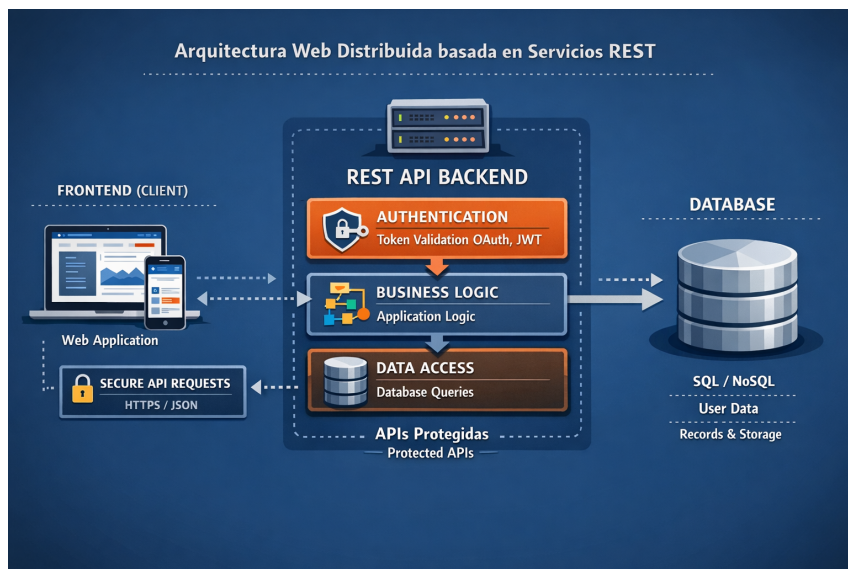


Figura 1: Arquitectura web distribuida basada en servicios REST. El frontend consume APIs protegidas expuestas por el backend, el cual encapsula la lógica de negocio, los mecanismos de autenticación y el acceso a la base de datos.

III. Svelte: un paradigma diferente en el desarrollo frontend

Svelte introduce un enfoque distintivo en el desarrollo frontend al adoptar un modelo basado en compilación, en contraste con los frameworks tradicionales que dependen de bibliotecas de ejecución (*runtime*) en el navegador. Mientras que soluciones como React o Vue gestionan la actualización de la interfaz mediante técnicas de reconciliación sobre un Virtual DOM, Svelte traslada esta responsabilidad al proceso de construcción de la

aplicación, generando código JavaScript optimizado que opera directamente sobre el DOM [5, 8].

Desde una perspectiva arquitectónica, este enfoque resulta especialmente relevante en aplicaciones que consumen servicios web REST. Al reducir la sobrecarga en tiempo de ejecución, el frontend puede concentrarse en la gestión eficiente de estados derivados de la comunicación con el backend, tales como solicitudes asíncronas, recepción de datos, manejo de errores y validación de permisos. Esto favorece interfaces más predecibles y ligeras, lo cual es deseable en escenarios donde el acceso a los datos depende completamente de APIs externas [9].

El sistema de reactividad integrado de Svelte permite que los cambios en los datos obtenidos desde servicios web se reflejen automáticamente en la interfaz de usuario sin necesidad de mecanismos adicionales como observadores explícitos o APIs complejas [10]. Esta característica facilita la implementación de patrones comunes en aplicaciones cliente-servidor, como la actualización dinámica de listas tras operaciones CRUD o la adaptación inmediata de la interfaz ante respuestas de error provenientes del backend [5].

En el contexto del control de acceso, Svelte actúa como consumidor de servicios protegidos, integrándose con mecanismos de autenticación basados en tokens, como JWT. Aunque la validación de credenciales y permisos recae exclusivamente en el backend, el frontend debe gestionar el estado de autenticación del usuario y adaptar la interfaz según los roles asignados. La reactividad de Svelte simplifica esta tarea, permitiendo que cambios en el estado de autenticación se propaguen de manera automática a los componentes de la aplicación.

La Figura 2 ilustra la interacción entre un frontend desarrollado con Svelte y un backend basado en Spring Boot, destacando el flujo de solicitudes autenticadas y la actualización reactiva de la interfaz en función de las respuestas del servidor.



Figura 2: Interacción entre frontend Svelte y backend Spring Boot. El frontend consume servicios REST protegidos mediante tokens de autenticación y actualiza la interfaz de forma reactiva según las respuestas del servidor.

III.I. Modelo de componentes en Svelte

A diferencia de frameworks como React o Vue que estructuran componentes mediante funciones o clases con métodos específicos del ciclo de vida, Svelte utiliza archivos con extensión `.svelte` que combinan markup, lógica y estilos en una única unidad cohesiva. Esta arquitectura de archivo único simplifica la organización del código y facilita la comprensión de componentes autónomos que consumen servicios REST.

En un componente Svelte orientado al consumo de servicios REST, la estructura se organiza alrededor de tres elementos fundamentales: la definición del estado local, la ejecución de solicitudes al backend y la representación condicional de la interfaz. El estado del componente se declara mediante variables reactivas que almacenan los datos obtenidos, así como indicadores de carga y error asociados a la comunicación con el servidor. Las peticiones HTTP suelen ejecutarse durante la fase de inicialización del componente, permitiendo que los datos sean recuperados una vez que la vista ha sido renderizada.

La representación de la interfaz se adapta dinámicamente al estado del componente mediante estructuras declarativas, lo que posibilita mostrar mensajes de carga, notificaciones de error o los datos recibidos según corresponda. Este modelo de organización favorece la encapsulación de la lógica de presentación y simplifica el mantenimiento de componentes que dependen de servicios web externos, manteniendo una clara separación entre la obtención de datos y su visualización [5].

III.II. Reactividad declarativa y actualización eficiente del DOM

La reactividad es el mecanismo fundamental que permite a Svelte sincronizar automáticamente el estado de la aplicación con la interfaz de usuario. A diferencia de frameworks que requieren llamadas explícitas a funciones como `setState` o el uso de proxies reactivos, Svelte implementa la reactividad mediante un análisis estático del código durante la compilación, generando instrucciones de actualización específicas para cada dependencia detectada [10]. En Svelte, la reactividad declarativa permite que las variables dependientes se actualicen automáticamente cuando cambia el estado de la aplicación. Este comportamiento se logra mediante declaraciones reactivas que expresan relaciones directas entre los datos, evitando la necesidad de invocar métodos explícitos para forzar la actualización de la interfaz. Como resultado, cualquier modificación en los datos obtenidos desde servicios REST se propaga de manera inmediata a los elementos visuales que dependen de ellos.

Este modelo resulta especialmente adecuado para escenarios comunes en aplicaciones cliente-servidor, como el filtrado, ordenamiento o transformación de colecciones de datos recibidas desde el backend. Al delegar la sincronización entre estado y vista al compilador, Svelte reduce la complejidad del código y minimiza errores asociados a la gestión manual del DOM, contribuyendo a una experiencia de usuario más coherente y predecible [9, 10].

III.III. Gestión de estado en aplicaciones Svelte

Cuando múltiples componentes necesitan compartir información obtenida desde servicios REST, como el estado de autenticación o datos de usuario, Svelte proporciona un sistema de *stores* que centraliza el estado global de forma reactiva. Un store es un objeto que contiene un valor y notifica automáticamente a todos los componentes suscritos cuando dicho valor cambia.

Svelte ofrece tres tipos de stores integrados: **writable** (lectura y escritura), **readable** (solo lectura), y **derived** (valores calculados a partir de otros stores). Para aplicacio-

nes que consumen APIs REST, los stores `writable` resultan particularmente útiles para gestionar tokens de autenticación y datos de usuario.

En aplicaciones frontend que consumen servicios REST, la gestión del estado global resulta necesaria cuando múltiples componentes deben compartir información común, como datos de usuario autenticado o parámetros de sesión. En Svelte, este problema se aborda mediante un sistema de estado centralizado que permite almacenar y distribuir valores de forma reactiva entre los distintos componentes de la aplicación.

Este enfoque facilita la propagación automática de cambios en el estado global hacia todas las vistas que dependen de dicha información, evitando el paso manual de datos entre componentes y reduciendo el acoplamiento estructural. En escenarios de consumo de APIs REST, la gestión centralizada del estado contribuye a una organización más clara de la aplicación y a una mejor separación entre la lógica de acceso a datos y la lógica de presentación [6].

III.iv. Ciclo de vida y patrones de integración con servicios REST

Svelte proporciona funciones del ciclo de vida que permiten ejecutar código en momentos específicos de la existencia de un componente. Para el consumo de servicios REST, la función `onMount` resulta fundamental, ya que se ejecuta inmediatamente después de que el componente se renderiza en el DOM por primera vez, garantizando que las peticiones HTTP se realicen solo cuando el componente está efectivamente visible.

Además de `onMount`, Svelte ofrece `beforeUpdate` y `afterUpdate` para ejecutar código antes y después de actualizaciones del DOM, `onDestroy` para limpiar recursos cuando el componente se desmonta, y `tick` para esperar a que el siguiente ciclo de actualización del DOM se complete.

El ciclo de vida de los componentes en Svelte permite definir puntos específicos en los que se ejecuta lógica asociada a la inicialización, actualización y destrucción de una vista. En el contexto del consumo de servicios REST, estos mecanismos se utilizan para controlar el momento en que se realizan las solicitudes al backend y para asegurar que los recursos empleados durante la comunicación, como temporizadores o suscripciones, sean liberados adecuadamente cuando el componente deja de estar activo.

La correcta gestión del ciclo de vida resulta especialmente relevante en aplicaciones frontend que interactúan de forma intensiva con APIs externas, ya que previene la ejecución innecesaria de peticiones, evita fugas de memoria y contribuye a un comportamiento predecible de la interfaz. Este enfoque favorece una integración más robusta entre la capa de presentación y los servicios web, manteniendo la estabilidad del sistema incluso en escenarios de interacción dinámica o navegación frecuente [5].

IV. Consumo de servicios web REST con Svelte

El consumo de APIs REST constituye una operación fundamental en aplicaciones frontend modernas. Desde la perspectiva del frontend, Svelte proporciona un enfoque elegante y eficiente para gestionar las complejidades asociadas con peticiones asíncronas, manejo de estados de carga, procesamiento de respuestas y gestión de errores.

IV.1. Gestión de estados derivados de peticiones asíncronas

El consumo de servicios web REST desde el frontend implica necesariamente la ejecución de operaciones asíncronas, cuyo resultado no es inmediato y depende de factores como la latencia de red y la disponibilidad del servidor. En este contexto, una aplicación frontend debe gestionar explícitamente los estados asociados a cada solicitud, principalmente el estado de carga, el estado de éxito y el estado de error.

Svelte facilita este modelo de interacción mediante su sistema de reactividad integrado, el cual permite que los cambios en los datos obtenidos desde el backend se reflejen automáticamente en la interfaz de usuario. Cuando una solicitud HTTP es iniciada, el estado de la aplicación puede actualizarse para indicar que la operación está en progreso; una vez recibida la respuesta, la interfaz se adapta dinámicamente según el resultado de la operación. Este enfoque declarativo reduce la complejidad del código y mejora la mantenibilidad de la aplicación [5, 11].

La correcta gestión de estos estados resulta especialmente relevante en aplicaciones que dependen exclusivamente de servicios web para acceder a la información, ya que permite ofrecer retroalimentación clara al usuario y evitar inconsistencias visuales durante la interacción con el sistema.

IV.1.1. Gestión de estados asíncronos en interfaces reactivas

Svelte simplifica la implementación del patrón de gestión de estados asíncronos mediante el uso de variables reactivas que representan cada fase de la petición HTTP. Un patrón robusto para consumir APIs REST incluye el seguimiento de tres estados fundamentales: `loading` (carga en progreso), `data` (datos recibidos), y `error` (error capturado).

En interfaces reactivas que consumen servicios REST, la gestión explícita de los estados asíncronos permite representar de forma clara las distintas fases de una solicitud HTTP. Estos estados suelen contemplar la carga inicial de los datos, la visualización de la información recibida y la notificación de errores cuando la comunicación con el backend falla. La correcta representación de dichos estados contribuye a mejorar la experiencia de usuario y a evitar comportamientos inconsistentes durante la interacción con el sistema.

Este patrón resulta especialmente relevante en aplicaciones orientadas a la gestión de información, donde la disponibilidad de los datos depende completamente de servicios externos. Al definir de manera explícita los estados asociados a las peticiones, el frontend puede reaccionar de forma controlada ante escenarios de latencia, fallos de red o respuestas inesperadas del servidor, manteniendo la estabilidad de la interfaz [5]. Además, garantiza que la interfaz refleje con precisión el estado actual de la operación asíncrona. El bloque `#if/{:else if}/{:else}` de Svelte permite renderizar condicionalmente diferentes secciones de la interfaz según el estado de la petición, proporcionando retroalimentación inmediata al usuario y permitiendo la recuperación de errores mediante acciones explícitas como el botón de reintento [5].

IV.1.2. Renderizado basado en promesas y control de flujo asíncrono

Svelte proporciona una sintaxis especial para manejar promesas directamente en el markup mediante bloques `{await}`, que eliminan la necesidad de gestionar manualmente los estados de carga, éxito y error. Esta característica resulta particularmente útil para consumir APIs REST de forma declarativa:

El renderizado basado en promesas constituye un enfoque declarativo para gestionar el flujo asíncrono de datos en aplicaciones frontend. Mediante este patrón, la interfaz puede adaptarse automáticamente a los diferentes estados de una promesa asociada a una solicitud REST, mostrando contenido de carga mientras la operación se completa y reaccionando de forma diferenciada ante resultados exitosos o errores.

Este modelo reduce la necesidad de lógica imperativa adicional para el control de estados y favorece una separación más clara entre la obtención de datos y su presentación. En el contexto del consumo de servicios REST, el renderizado basado en promesas simplifica la construcción de interfaces reactivas y contribuye a una mayor legibilidad y mantenibilidad del código [5, 11].

IV.I.III. Control de concurrencia en consumo de APIs

En aplicaciones interactivas donde el usuario puede cambiar rápidamente de contexto (por ejemplo, navegando entre diferentes vistas o aplicando filtros), resulta fundamental implementar mecanismos de cancelación de peticiones para evitar *race conditions* donde respuestas de peticiones antiguas sobrescriben datos de peticiones más recientes. Svelte facilita esta gestión mediante el uso de `AbortController`.

En aplicaciones interactivas que realizan múltiples solicitudes a servicios REST, el control de concurrencia se vuelve un aspecto crítico para garantizar la consistencia de los datos mostrados en la interfaz. Cuando varias peticiones se ejecutan de forma simultánea o en rápida sucesión, existe el riesgo de que respuestas obsoletas sobrescriban información más reciente, generando inconsistencias visuales o lógicas.

La adopción de estrategias de control de concurrencia permite mitigar estos problemas, asegurando que solo las respuestas relevantes actualicen el estado de la aplicación. Este enfoque resulta especialmente útil en escenarios de búsqueda dinámica, filtrado o navegación frecuente, donde la interacción del usuario puede desencadenar múltiples solicitudes consecutivas al backend [5].

Este patrón garantiza que solo la respuesta de la petición más reciente actualiza la interfaz, cancelando automáticamente peticiones previas que aún no han completado. La función `onDestroy` asegura que cualquier petición pendiente se cancele cuando el componente se desmonta, previniendo actualizaciones de estado sobre componentes inexistentes [5].

IV.II. Consumo seguro de servicios REST

El consumo de servicios web REST desde Svelte se enmarca dentro del modelo de arquitectura cliente-servidor, en el cual el frontend actúa como consumidor de recursos expuestos por el backend a través de interfaces bien definidas. Este enfoque permite una clara separación de responsabilidades, favoreciendo la escalabilidad, el desacoplamiento y la mantenibilidad del sistema [1].

La comunicación segura entre el frontend y el backend se basa en mecanismos de autenticación mediante tokens, comúnmente implementados a través de JSON Web Tokens (JWT). Dichos tokens son obtenidos tras un proceso de autenticación exitoso y enviados en cada solicitud HTTP mediante el encabezado `Authorization`, permitiendo que el backend valide la identidad del usuario y controle el acceso a los recursos protegidos [7]. En este contexto, el frontend se limita a gestionar y transmitir el token, sin asumir responsabilidades de validación de seguridad.

Desde el lado del cliente, Svelte facilita el manejo eficiente de solicitudes asíncronas, estados de carga y actualización de la interfaz mediante su modelo de reactividad, lo que permite responder de forma inmediata a las respuestas del backend. Diversos estudios han demostrado que frameworks modernos como Svelte ofrecen un rendimiento competitivo y una gestión eficiente de recursos en aplicaciones web basadas en consumo de APIs REST [5].

V. Operaciones CRUD, consultas y visualización de datos en Svelte

El consumo de servicios REST desde aplicaciones frontend no se limita únicamente a procesos de autenticación, sino que constituye la base para la implementación de operaciones CRUD (Create, Read, Update, Delete), consultas y generación de reportes sobre los datos gestionados por el backend. En este contexto, Svelte actúa como una capa de presentación que interactúa de forma directa con los servicios REST, permitiendo la manipulación y visualización dinámica de la información [1].

V.I. Implementación de operaciones CRUD

Las operaciones CRUD se implementan mediante el uso de métodos HTTP estandarizados, tales como GET, POST, PUT y DELETE, los cuales permiten al frontend recuperar, crear, actualizar y eliminar registros respectivamente. Svelte facilita este proceso mediante el manejo de solicitudes asíncronas y la actualización reactiva de la interfaz, de modo que los cambios en los datos se reflejan automáticamente en la vista sin necesidad de recargar la página. Este enfoque es característico de los frameworks frontend modernos y ha demostrado un desempeño competitivo en aplicaciones basadas en consumo intensivo de APIs REST [5].

V.II. Tablas y consultas de datos

La visualización de datos en forma de tablas constituye un elemento fundamental en aplicaciones orientadas a la gestión de información. En Svelte, las tablas pueden construirse a partir de los datos obtenidos desde el backend, incorporando mecanismos de filtrado, ordenamiento y paginación controlados desde el frontend o delegados al servidor según el volumen de información. Diversos estudios sobre APIs REST destacan la importancia de un diseño adecuado de las consultas y de los parámetros de búsqueda para optimizar el rendimiento y la experiencia de usuario [12].

V.III. Generación de reportes

La generación de reportes en aplicaciones web puede abordarse desde dos enfoques principales: la generación en el backend, donde el servidor produce archivos en formatos como PDF o Excel, y la generación en el frontend, donde los datos obtenidos mediante servicios REST son procesados y presentados dinámicamente. En ambos casos, el acceso a los datos utilizados para los reportes debe estar protegido por los mismos mecanismos de autenticación y autorización que regulan las operaciones CRUD, garantizando la confidencialidad y la integridad de la información [7].

Desde la perspectiva del frontend, Svelte cumple un rol clave en la presentación y consumo de la información utilizada para la generación de reportes. A través del consumo de servicios REST, la aplicación puede solicitar conjuntos de datos agregados o filtrados y representarlos dinámicamente mediante tablas, gráficos u otros componentes visuales, permitiendo al usuario interactuar con la información sin recargar la vista. De este modo, Svelte actúa como intermediario entre los servicios de reporte del backend y la visualización final de los resultados.

VI. Buenas prácticas en el consumo de servicios REST desde Svelte

El desarrollo de aplicaciones web basadas en consumo de servicios REST requiere la adopción de buenas prácticas que garanticen mantenibilidad, seguridad y un desempeño adecuado. En el contexto de aplicaciones frontend construidas con Svelte, estas prácticas permiten una integración eficiente con el backend y una correcta gestión de las operaciones CRUD, consultas y reportes.

En primer lugar, resulta fundamental mantener una clara separación de responsabilidades entre el frontend y el backend, delegando la lógica de negocio, la validación de datos y los mecanismos de seguridad exclusivamente al servidor. Este enfoque es coherente con los principios de diseño REST y favorece la escalabilidad del sistema [1].

Asimismo, el consumo de servicios debe centralizarse mediante capas o módulos de acceso a datos en el frontend, lo que evita la duplicación de lógica y facilita el mantenimiento del código. El uso de métodos HTTP estandarizados y respuestas bien definidas contribuye a una comunicación clara entre los componentes del sistema y reduce errores en la interacción cliente-servidor [12].

Desde el punto de vista de la seguridad, es indispensable aplicar mecanismos de autenticación y autorización de forma consistente en todas las operaciones, incluyendo la generación de reportes. El uso de tokens JWT y la correcta protección de los endpoints permiten controlar el acceso a los recursos y preservar la confidencialidad de la información [7].

Finalmente, el aprovechamiento del modelo de reactividad de Svelte facilita la gestión de estados de carga, error y actualización de la interfaz, mejorando la experiencia de usuario y permitiendo que los cambios en los datos se reflejen de manera inmediata en la vista [5].

VII. Conclusiones

El análisis realizado permitió evidenciar que el uso de arquitecturas basadas en servicios REST constituye una base sólida para el desarrollo de aplicaciones web modernas, al favorecer la separación de responsabilidades entre frontend y backend, así como la escalabilidad y mantenibilidad del sistema. Este enfoque resulta especialmente adecuado en escenarios donde el frontend actúa como consumidor de servicios para la gestión de operaciones CRUD, consultas y generación de reportes. La adopción de Svelte como framework frontend demuestra ser una alternativa eficiente para la construcción de interfaces reactivas, permitiendo una actualización dinámica de la vista en función de las respuestas del backend. Su modelo de reactividad y bajo consumo de recursos facilita la implemen-

tación de tablas, formularios y visualizaciones de datos de manera clara y performante, sin introducir una complejidad innecesaria en la capa de presentación.

Asimismo, se destaca la importancia de integrar mecanismos de seguridad de forma transversal en el consumo de servicios REST, aplicando autenticación y autorización basadas en tokens para proteger tanto las operaciones CRUD como el acceso a reportes. La centralización de la lógica de seguridad en el backend contribuye a un control más robusto del sistema y reduce el acoplamiento con el frontend.

En conjunto, los elementos analizados permiten concluir que la combinación de servicios REST, un frontend moderno como Svelte y la aplicación de buenas prácticas en el consumo de APIs constituye una solución adecuada para el desarrollo de aplicaciones web orientadas a la gestión de información, garantizando un equilibrio entre rendimiento, seguridad y experiencia de usuario.

Referencias

- [1] C. Pautasso, “RESTful Web Services: Principles, Patterns, Emerging Technologies,” en *Web Services Foundations*, A. Bouguettaya, Q. Z. Sheng y F. Daniel, eds., New York, NY: Springer, 2014, págs. 31-51, ISBN: 978-1-4614-7518-7. DOI: 10.1007/978-1-4614-7518-7_2.
- [2] Y. Xu, G. Bai, W. Xiao, R. Li y F. Zeng, “RESTful API Service Discovery via Comprehensive Feature Mining, Deep Neural Networks, and Contrastive Learning,” en *Service-Oriented Computing – ICSOC 2025*, M. Aiello, S. Deng, J. M. Murillo, I. Georgievski, B. Benatallah y Z. Wang, eds., ép. Lecture Notes in Computer Science, vol. 16320, Singapore: Springer, 2026, págs. 19-36, ISBN: 978-981-95-5012-8. DOI: 10.1007/978-981-95-5012-8_2.
- [3] S. R. Modugu y H. Farhat, “Implementation of the Internet of Things Application Based on Spring Boot Microservices and REST Architecture,” en *Software Engineering Perspectives in Intelligent Systems*, R. Silhavy, P. Silhavy y Z. Prokopova, eds., ép. Advances in Intelligent Systems and Computing, vol. 1294, Cham: Springer, 2020, págs. 27-38, ISBN: 978-3-030-63322-6. DOI: 10.1007/978-3-030-63322-6_3.
- [4] S. Di Meglio, V. Pontillo y L. L. L. Starace, “REST in Pieces: RESTful Design Rule Violations in Student-Built Web Apps,” en *Software Engineering and Advanced Applications*, D. Taibi y D. Smite, eds., ép. Lecture Notes in Computer Science, vol. 16320, Cham: Springer, 2026, págs. 123-138, ISBN: 978-3-032-04207-1. DOI: 10.1007/978-3-032-04207-1_13.
- [5] F. P. E. Putra, H. Hasbullah, F. Muslim y R. Paradina, “Technical Performance Comparison of Modern Frontend Frameworks Study on Svelte, React, and Vue,” *Brilliance: Research of Artificial Intelligence*, vol. 5, n.º 1, págs. 355-364, mayo de 2025, ISSN: 2807-9035. DOI: 10.47709/brilliance.v5i1.6133.
- [6] S. Karlsson, A. Causevic y D. Sundmark, “Exploring Behaviours of RESTful APIs in an Industrial Setting,” *Software Quality Journal*, vol. 32, n.º 3, págs. 897-928, jul. de 2024, ISSN: 1573-1367. DOI: 10.1007/s11219-024-09686-0.
- [7] M. Nardone, “JSON Web Token (JWT) Authentication,” en *Secure RESTful APIs: Simple Solutions for Beginners*, Berkeley, CA: Apress, 2025, págs. 45-89, ISBN: 979-8-8688-1285-9. DOI: 10.1007/979-8-8688-1285-9_4.

-
- [8] S. Di Meglio y L. L. L. Starace, “Evaluating Performance and Resource Consumption of REST Frameworks and Execution Environments: Insights and Guidelines for Developers and Companies,” *IEEE Access*, vol. 12, págs. 161 649-161 669, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3489892.
 - [9] K. S. S. Kumar, “Reactive Programming Paradigms in High-Throughput Distributed Systems,” *European Journal of Computer Science and Information Technology*, vol. 13, n.º 30, págs. 93-103, mayo de 2025, ISSN: 2054-0957. DOI: 10.37745/ejcsit.2013/vol13n3093103.
 - [10] E. Bainomugisha, A. L. Carreton, T. V. Cutsem, S. Mostinckx y W. D. Meuter, “A Survey on Reactive Programming,” *ACM Computing Surveys*, vol. 45, n.º 4, págs. 1-34, ago. de 2013, ISSN: 0360-0300. DOI: 10.1145/2501654.2501666.
 - [11] A. Zbarcea, “Extending the Migration from Asynchronous to Reactive Programming in Java: A Performance Analysis of Caching, CPU-Bound, and Blocking Scenarios,” *Applied Sciences*, vol. 16, n.º 1, pág. 90, ene. de 2026, ISSN: 2076-3417. DOI: 10.3390/app16010090.
 - [12] A. Neumann, N. Laranjeiro y J. Bernardino, “An Analysis of Public REST Web Service APIs,” *IEEE Transactions on Services Computing*, vol. 14, n.º 4, págs. 957-970, jul. de 2021, ISSN: 1939-1374. DOI: 10.1109/TSC.2018.2847344.

Manual técnico

Manual para aplicación de Svelte

1. Descripción del manual

El presente manual técnico describe la implementación de una aplicación frontend utilizando el framework Svelte para el consumo e integración de servicios web REST. El objetivo del manual es proporcionar una guía práctica y reutilizable que permita desarrollar interfaces web capaces de interactuar con un backend mediante operaciones de gestión de datos, consultas dinámicas, visualización interactiva de información y generación de reportes.

Este manual está diseñado con un enfoque general, de modo que los ejemplos y componentes presentados puedan ser aplicados a distintos tipos de sistemas web, independientemente de su dominio específico. Las funcionalidades descritas incluyen operaciones CRUD (Create, Read, Update, Delete), ejecución de consultas dinámicas, construcción de tablas interactivas y generación de reportes basados en datos obtenidos desde servicios REST. Estas capacidades constituyen elementos fundamentales en aplicaciones modernas orientadas a la gestión de información.

La estructura del manual se organiza en cuatro ejes principales. En primer lugar, se presenta la implementación de operaciones CRUD que permiten la creación, consulta, actualización y eliminación de registros mediante la integración con APIs REST. En segundo lugar, se describe el desarrollo de mecanismos de consultas dinámicas que permiten filtrar, buscar y recuperar información de forma eficiente. En tercer lugar, se detalla la construcción de tablas dinámicas que visualizan datos de forma reactiva y permiten mejorar la interacción del usuario con la información. Finalmente, se aborda la generación de reportes mediante el procesamiento y presentación de datos provenientes del backend.

Cada sección del manual incluye ejemplos funcionales que ilustran la implementación de estas funcionalidades utilizando Svelte, aplicando buenas prácticas de desarrollo frontend, separación de responsabilidades y consumo estructurado de servicios REST. Los ejemplos presentados son independientes entre sí y pueden adaptarse a distintos proyectos, permitiendo su reutilización en diferentes contextos de desarrollo.

1.1. Requisitos previos y configuración inicial

Antes de implementar las funcionalidades descritas en este manual, es necesario contar con un entorno de desarrollo adecuado para aplicaciones frontend modernas. Se requiere tener instalado Node.js versión 18 o superior junto con npm, el cual permite gestionar dependencias y ejecutar herramientas de desarrollo.

El proyecto debe inicializarse utilizando Vite como herramienta de construcción, empleando la plantilla oficial de Svelte. Esta configuración proporciona un entorno optimizado que permite el desarrollo de aplicaciones reactivas, con soporte para módulos, recarga automática y compilación eficiente del código.

Adicionalmente, pueden utilizarse librerías complementarias que facilitan la implementación de funcionalidades comunes en aplicaciones web. Por ejemplo, la librería `svelte-routing` permite gestionar la navegación entre vistas, `date-fns` facilita el manejo y formateo de fechas, y `yup` permite realizar validación estructurada de datos en formularios.

Para garantizar una adecuada organización del proyecto, se recomienda estructurar el código en directorios separados según su función. Esta organización facilita el mantenimiento, mejora la escalabilidad y permite una clara separación entre los distintos componentes de la aplicación.

1.2. Estructura del proyecto Svelte

La estructura del proyecto se organiza en directorios que agrupan los archivos según su responsabilidad dentro de la arquitectura del sistema. El directorio **components** contiene los componentes de interfaz de usuario, los cuales implementan la lógica visual y la interacción con el usuario. Estos componentes pueden representar formularios, tablas, vistas de consulta u otros elementos funcionales de la aplicación.

El directorio **services** contiene los módulos encargados de la comunicación con el backend, encapsulando las solicitudes HTTP necesarias para consumir los servicios REST. Esta capa permite centralizar el acceso a los datos y evita la duplicación de lógica en distintos componentes.

El directorio **stores** se utiliza para gestionar el estado global de la aplicación mediante el sistema de reactividad de Svelte. Esta funcionalidad permite compartir datos entre múltiples componentes de forma consistente y reactiva.

Finalmente, el directorio **utils** contiene funciones auxiliares que pueden utilizarse en distintas partes del sistema, como validación de datos, formateo de información o transformación de resultados.

Esta estructura modular favorece el desarrollo organizado, facilita la reutilización de código y permite extender la aplicación de forma mantenible y escalable.

1.3. Configuración del servicio base de API

El consumo de servicios REST requiere una capa de abstracción que centralice la configuración de las solicitudes HTTP, el manejo de autenticación y la gestión de errores. Esta capa se implementa mediante un servicio base que define los métodos necesarios para comunicarse con el backend.

El servicio base utiliza variables de entorno para definir la URL del servidor, lo que permite adaptar la configuración según el entorno de ejecución, como desarrollo, pruebas o producción. Además, este servicio puede incorporar automáticamente encabezados de autenticación, como tokens, cuando sea necesario, permitiendo el acceso a recursos protegidos.

Esta arquitectura permite desacoplar la lógica de comunicación con el backend de los componentes de interfaz, mejorando la mantenibilidad del código y facilitando la reutilización de los servicios en distintos módulos de la aplicación.

2. Operaciones CRUD

Las operaciones CRUD (Create, Read, Update, Delete) representan la base de la gestión de información en el Sistema de Selección Docente, ya que permiten administrar entidades del dominio tales como facultades, carreras, convocatorias, postulaciones y demás recursos que el backend expone mediante servicios REST.

En el frontend desarrollado con SvelteKit, estas operaciones se implementan aplicando separación de responsabilidades: (i) una configuración de proxy para facilitar el consumo de la API en desarrollo, (ii) una capa de acceso HTTP reutilizable (`apiFetch`) que centraliza el manejo de errores y respuestas, (iii) una capa de servicios que define operaciones por recurso (por ejemplo, `facultadService`) y (iv) componentes Svelte que gestionan el estado local y la interacción del usuario.

2.1. Proxy de desarrollo en Vite/SvelteKit

Durante el desarrollo local del Sistema de Selección Docente, el frontend se ejecuta en un puerto diferente al backend. Para evitar problemas de CORS y para simplificar las URLs de consumo, se configura un proxy de desarrollo que redirige todas las solicitudes iniciadas en `/api` hacia el backend.

```

1 import { sveltekit } from '@sveltejs/kit/vite';
2 import { defineConfig } from 'vite';
3
4 export default defineConfig({
5   plugins: [sveltekit()],
6   server: {
7     proxy: {
8       '/api': {
9         target: 'http://localhost:8080',
10        changeOrigin: true
11      }
12    }
13  }
14 });

```

Listing 1: Proxy de desarrollo para consumir la API del Sistema de Selección Docente

Esta configuración permite que el frontend invoque endpoints como `/api/facultades` sin incluir explícitamente `http://localhost:8080`. Además, facilita pruebas rápidas del flujo CRUD mientras se implementan módulos del sistema (por ejemplo, mantenimiento de catálogos como facultades y carreras).

2.2. Capa base de comunicación HTTP (`apiFetch`)

Para evitar la repetición de código en cada componente, se implementa una función genérica `apiFetch<T>` que se encarga de: (i) enviar peticiones HTTP con cabeceras JSON, (ii) interpretar respuestas exitosas, (iii) transformar errores del backend en un objeto consistente (`ApiError`) y (iv) manejar respuestas 204 `No Content`, comunes en operaciones DELETE.


```

1 export type ApiError={
2     status?: number;
3     error?: string;
4     message?: string;
5     path?: string;
6     timestamp?: string;
7 }
8
9 async function parseError(res: Response): Promise<ApiError>{
10     try{
11         const data = await res.json();
12         return data?? {status: res.status, message: res.
13             statusText}
14     } catch {
15         return {
16             status: res.status, message: res.statusText
17         };
18     }
19 }
20
21 export async function apiFetch <T>(  
22     url: string,  
23     options: RequestInit = {}  
24 ): Promise <T> {  
25     const res = await fetch(url, {  
26         ...options,  
27         headers: {  
28             'Content-Type': 'application/json',  
29             ...(options.headers ?? {})  
30         }  
31     });  
32     if(!res.ok)  
33     {  
34         const err = await parseError(res);  
35         throw err;  
36     }  
37
38     if(res.status === 204){  
39         return undefined as T;  
40     }  
41
42     return (await res.json()) as T;  
43 }

```

Listing 2: Función base apiFetch con manejo consistente de errores

En el contexto del Sistema de Selección Docente, esta capa permite que los módulos del frontend (por ejemplo, gestión de facultades o carreras) obtengan un manejo de errores uniforme. Esto facilita mostrar al usuario mensajes claros ante errores de validación, conflictos o fallos de conexión, manteniendo el código de la interfaz más limpio.

2.3. Modelado de datos con TypeScript

En el frontend se definen tipos que representan el contrato de datos con el backend. Esto es especialmente útil en un sistema académico porque permite mantener coherencia entre nombres de atributos (por ejemplo, `idFacultad`, `nombreFacultad`, `estado`) y reduce errores al consumir y mostrar información.

```
1 export type Facultad = {  
2     idFacultad: number;  
3     nombreFacultad: string;  
4     estado: boolean;  
5 };  
6  
7 export type Carrera = {  
8     idCarrera: number;  
9     idFacultad: number;  
10    nombreCarrera: string;  
11    modalidad: string;  
12    estado: boolean;  
13 };
```

Listing 3: Tipos de dominio utilizados en el módulo de catálogos del sistema

En el Sistema de Selección Docente, estos catálogos suelen alimentar formularios y filtros posteriores (por ejemplo, seleccionar facultad/carrera al registrar información de convocatorias u otros procesos).

2.4. Servicios por recurso: ejemplo con facultades

La capa de servicios encapsula las operaciones CRUD de cada recurso del sistema, evitando que los componentes manejen rutas, métodos HTTP y serialización manual. El servicio mostrado implementa: listar, crear, actualizar, cambiar estado (activación/desactivación) y eliminar.

```

1 import { apiFetch } from "$lib/utils/apiFetch";
2 import type { Facultad } from "$lib/types";
3
4 export const facultadService = {
5   list: () => apiFetch<Facultad[]>('api/facultades'),
6
7   create: (nombreFacultad: string) =>
8     apiFetch<Facultad>('api/facultades', {
9       method: 'POST',
10      body: JSON.stringify({ nombreFacultad })
11    }),
12
13   update: (id: number, nombreFacultad: string) =>
14     apiFetch<Facultad>('api/facultades/${id}', {
15       method: 'PUT',
16       body: JSON.stringify({ nombreFacultad })
17     }),
18
19   setEstado : (id: number, estado: boolean) =>
20     apiFetch<Facultad>('api/facultades/${id}/estado', {
21       method: 'PATCH',
22       body: JSON.stringify({ estado })
23     }),
24
25   remove: (id: number) =>
26     apiFetch<void>('api/facultades/${id}', {
27       method: 'DELETE'
28     }),
29 };

```

Listing 4: Servicio REST para el recurso Facultades (CRUD completo)

La operación `setEstado` resulta útil en sistemas institucionales, ya que permite desactivar registros sin eliminarlos físicamente, manteniendo trazabilidad y evitando pérdida de datos históricos.

2.5. Componente Svelte: interfaz CRUD

El componente de interfaz implementa el mantenimiento de Facultades del Sistema de Selección Docente. Para mantener el código organizado, se divide en: (i) definición de estado y carga inicial, (ii) preparación del formulario, (iii) operaciones de persistencia (crear/editar/estado/eliminar) y (iv) renderizado de la vista.

2.5.1. Estado del componente y carga inicial

En primer lugar, se declaran variables locales para almacenar los datos, el error y el estado del formulario. Luego, se usa `onMount` para cargar los datos apenas el componente se renderiza por primera vez.

```
1 <script lang="ts">
2   import { onMount } from 'svelte';
3   import type { Facultad } from '$lib/types';
4   import type { ApiError } from '$lib/utils/apiFetch';
5   import { facultadService } from '$lib/services/facultadService';
6
7   let facultades: Facultad[] = [];
8   let error: ApiError | null = null;
9
10  let nombre = '';
11  let editId: number | null = null;
12
13  async function cargar(){
14    error = null;
15    try{
16      facultades = await facultadService.list();
17    }catch (e){
18      error = e as ApiError;
19    }
20  }
21
22  onMount(cargar);
23 </script>
```

Listing 5: Estado local y carga inicial con `onMount`

2.5.2. Preparación del formulario: crear y editar

Para diferenciar creación y edición, se usa `editId`. Si es `null` se crea; si tiene un valor, se edita el registro correspondiente. Además, se limpian campos y errores al iniciar cada operación.

```

1 function iniciarCrear(){
2   editId = null;
3   nombre = '';
4   error = null;
5 }
6
7 function iniciarEditar(f: Facultad){
8   editId = f.idFacultad;
9   nombre = f.nombreFacultad;
10  error = null;
11 }

```

Listing 6: Funciones para iniciar creación y edición

2.5.3. Operaciones: guardar, cambiar estado y eliminar

La función `guardar` valida el nombre en el cliente y decide entre `create` o `update`. Luego recarga la lista para reflejar cambios. Por su parte, `cambiarEstado` alterna el atributo `estado` (activo/inactivo) y `eliminar` solicita confirmación antes de ejecutar el `DELETE`.

```

1 async function guardar(){
2   error = null;
3   const n = nombre.trim();
4   if(!n){
5     error = { status: 400, message: 'El nombre es obligatorio' };
6     return;
7   }
8
9   try{
10    if(editId === null){
11      await facultadService.create(n);
12    }else{
13      await facultadService.update(editId, n);
14    }
15    iniciarCrear();
16    await cargar();
17  }catch(e){
18    error = e as ApiError;
19  }
20 }
21
22 async function cambiarEstado(f: Facultad){
23   error = null;
24   try{
25     await facultadService.setEstado(f.idFacultad, !f.estado);
26     await cargar();

```

```

27     }catch(e){
28         error = e as ApiError;
29     }
30 }
31
32 async function eliminar(f: Facultad){
33     error = null;
34     if(!confirm('Desea eliminar ${f.nombreFacultad}?')) return;
35
36     try{
37         await facultadService.remove(f.idFacultad);
38         await cargar();
39     }catch(e){
40         error = e as ApiError;
41     }
42 }

```

Listing 7: Guardar (crear/editar)

2.5.4. Renderizado: formulario, mensajes y tabla dinámica

Finalmente, la vista se compone de: un campo de entrada para el nombre, botones de acción, mensajes de error y una tabla dinámica que se renderiza con { #each }. Cuando no existen datos, se muestra un mensaje dentro de la tabla.

```

1 <h3>Facultades</h3>
2
3 <input placeholder="Nombre de facultad" bind:value={nombre} />
4
5 <button on:click={guardar}>
6     {editId === null ? 'Crear' : 'Guardar'}
7 </button>
8
9 {#if editId !== null}
10     <button on:click={iniciarCrear}>Cancelar</button>
11 {/if}
12
13 <button on:click={cargar}>Refrescar</button>
14
15 {#if error}
16     <p class="error">Error {error.status}: {error.message}</p>
17 {/if}
18
19 <table>
20     <thead>
21         <tr>
22             <th>ID</th>
23             <th>Nombre</th>
24             <th>Estado</th>
25             <th>Acciones</th>
26         </tr>
27     </thead>

```

```
28
29 <tbody>
30   {#each facultades as f}
31     <tr>
32       <td>{f.idFacultad}</td>
33       <td>{f.nombreFacultad}</td>
34       <td>{f.estado ? 'Activa' : 'Inactiva'}</td>
35       <td class="actions">
36         <button on:click={() => iniciarEditar(f)}>Editar</
           button>
37         <button on:click={() => cambiarEstado(f)}>
           {f.estado ? 'Desactivar' : 'Activar'}
38         </button>
39         <button on:click={() => eliminar(f)}>Eliminar</button>
40       </td>
41     </tr>
42   {/each}
43
44   {#if facultades.length === 0}
45     <tr>
46       <td colspan="4">No hay datos</td>
47     </tr>
48   {/if}
49 </tbody>
50 </table>
```

Listing 8: Vista del módulo: formulario

Este diseño permite mantener un flujo CRUD completo: carga inicial, creación, edición, activación/desactivación lógica y eliminación. La recarga posterior a cada operación garantiza que la tabla refleje el estado actual del backend.

2.5.5. Evidencia visual del módulo

La Figura 3 presenta evidencia del funcionamiento del módulo de mantenimiento de Facultades. Se recomienda que la captura incluya registros de prueba y muestre al menos una fila con acciones visibles, de modo que se evidencie la interacción CRUD y el control de estado.

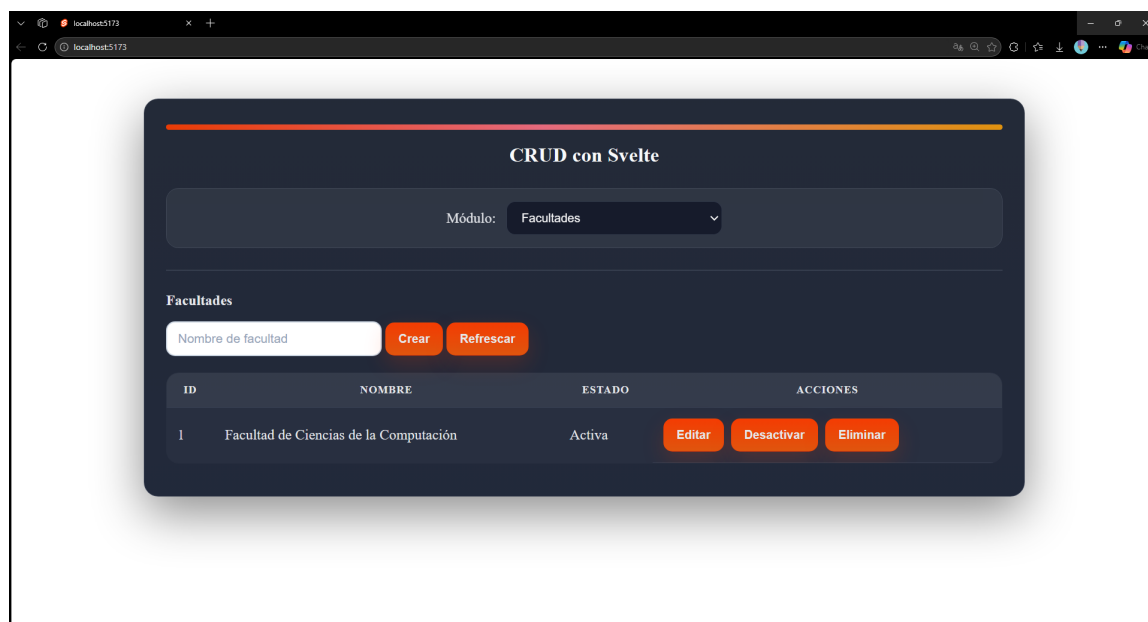


Figura 3: Interfaz del módulo de mantenimiento de Facultades del Sistema de Selección Docente. Se observa la tabla de registros, acciones CRUD y control de estado activo/i-nactivo.

3. Consultas y filtros dinámicos

La capacidad de buscar, filtrar y consultar información constituye una funcionalidad esencial en aplicaciones de gestión de datos. En el Sistema de Selección de Docentes, los usuarios necesitan localizar convocatorias específicas, buscar postulantes según criterios diversos y filtrar evaluaciones por estado o calificación. La implementación de estas funcionalidades puede realizarse mediante filtrado en el cliente, donde todos los datos se cargan inicialmente y se procesan localmente, o mediante consultas al servidor, donde el backend aplica los filtros y retorna únicamente los resultados relevantes.

La elección entre filtrado del lado del cliente o del servidor depende del volumen de datos y los requisitos de rendimiento. Para conjuntos de datos pequeños o medianos, el filtrado en el cliente proporciona una experiencia más fluida al evitar peticiones adicionales al servidor. Para conjuntos de datos grandes, el filtrado en el servidor resulta más eficiente al reducir la cantidad de información transferida y procesada en el navegador.

3.1. Filtrado del lado del cliente

El filtrado del lado del cliente aprovecha el sistema de reactividad de Svelte para actualizar la interfaz automáticamente cuando cambian los criterios de búsqueda. Este enfoque carga todos los datos disponibles una sola vez y aplica los filtros mediante transformaciones sobre el array de datos en memoria.

3.1.1. Implementación de búsqueda por texto

La búsqueda por texto permite a los usuarios localizar recursos ingresando términos que coincidan con campos específicos. La implementación reactiva en Svelte actualiza los resultados instantáneamente mientras el usuario escribe.

```

1 <!-- src/components/convocatorias/ConvocatoriasSearch.svelte -->
2 <script>
3   import { onMount } from 'svelte';
4   import { convocatoriasStore } from
5     '../stores/convocatoriasStore';
6   import { convocatoriasService } from
7     '../services/convocatoriasService';
8
9   let searchTerm = '';
10
11   $: filteredConvocatorias = filterConvocatorias(
12     $convocatoriasStore.convocatorias,
13     searchTerm
14   );
15
16   onMount(async () => {
17     await convocatoriasService.getAllConvocatorias();
18   });
19
20   function filterConvocatorias(convocatorias, term) {
21     if (!term.trim()) {
22       return convocatorias;
23     }

```

```

24     const lowerTerm = term.toLowerCase();
25
26
27     return convocatorias.filter(conv => {
28         const materia = conv.solicitud?.materia?.nombreMateria
29             ?.toLowerCase() || '';
30         const carrera = conv.solicitud?.materia?.carrera
31             ?.nombreCarrera?.toLowerCase() || '';
32         const area = conv.solicitud?.areaConocimiento
33             ?.nombreArea?.toLowerCase() || '';
34
35         return materia.includes(lowerTerm) ||
36             carrera.includes(lowerTerm) ||
37             area.includes(lowerTerm);
38     });
39 }
40 </script>
41
42 <div class="search-container">
43     <input
44         type="text"
45         bind:value={searchTerm}
46         placeholder="Buscar por materia, carrera o area..."
47     />
48
49     <p>
50         Mostrando {filteredConvocatorias.length} de
51         {$convocatoriasStore.convocatorias.length} convocatorias
52     </p>
53
54     {#if filteredConvocatorias.length === 0}
55         <p>No se encontraron resultados para "{searchTerm}"</p>
56     {:else}
57         <div class="results-list">
58             {#each filteredConvocatorias as convocatoria}
59                 <div class="convocatoria-item">
60                     <h3>
61                         {convocatoria.solicitud?.materia?.nombreMateria}
62                     </h3>
63                     <p>{convocatoria.solicitud?.materia?.carrera
64                         ?.nombreCarrera}</p>
65                     <p>{convocatoria.solicitud?.areaConocimiento
66                         ?.nombreArea}</p>
67                 </div>
68             {/each}
69         </div>
70     {/if}
71 </div>

```

Listing 9: Componente de búsqueda de convocatorias

La declaración reactiva \$: garantiza que la función de filtrado se ejecute automática-

mente cada vez que cambia el término de búsqueda o la lista de convocatorias, manteniendo los resultados sincronizados con el estado actual.

3.1.2. Filtros combinados múltiples

Los filtros múltiples permiten refinar los resultados aplicando varios criterios simultáneamente. Cada filtro se representa mediante una variable reactiva, y la función de filtrado combina todos los criterios activos.

```

1 <!-- src/components/convocatorias/ConvocatoriasAdvancedFilter.
   svelte -->
2 <script>
3   import { convocatoriasStore } from
4     '../stores/convocatoriasStore';
5
6   let searchTerm = '';
7   let selectedEstado = 'todas';
8   let selectedFacultad = null;
9   let facultades = [];
10
11   $: filteredConvocatorias = applyFilters(
12     $convocatoriasStore.convocatorias,
13     searchTerm,
14     selectedEstado,
15     selectedFacultad
16   );
17
18   function applyFilters(convocatorias, term, estado, facultad) {
19     let filtered = [...convocatorias];
20
21     // Filtrar por termino de busqueda
22     if (term.trim()) {
23       const lowerTerm = term.toLowerCase();
24       filtered = filtered.filter(conv =>
25         conv.solicitud?.materia?.nombreMateria
26           ?.toLowerCase().includes(lowerTerm) ||
27         conv.solicitud?.areaConocimiento?.nombreArea
28           ?.toLowerCase().includes(lowerTerm)
29       );
30     }
31
32     // Filtrar por estado
33     if (estado !== 'todas') {
34       filtered = filtered.filter(conv =>
35         conv.estado === estado
36       );
37     }
38
39     // Filtrar por facultad
40     if (facultad) {
41       filtered = filtered.filter(conv =>
42         conv.solicitud?.materia?.carrera?.facultad

```

```

43         ?.idFacultad === parseInt(facultad)
44     );
45 }
46
47     return filtered;
48 }
49
50     function clearFilters() {
51         searchTerm = '';
52         selectedEstado = 'todas';
53         selectedFacultad = null;
54     }
55 </script>
56
57 <div class="filters-panel">
58     <div class="filter-group">
59         <label>Buscar:</label>
60         <input
61             type="text"
62             bind:value={searchTerm}
63             placeholder="Materia o area de conocimiento"
64         />
65     </div>
66
67     <div class="filter-group">
68         <label>Estado:</label>
69         <select bind:value={selectedEstado}>
70             <option value="todas">Todas</option>
71             <option value="abierta">Abiertas</option>
72             <option value="cerrada">Cerradas</option>
73         </select>
74     </div>
75
76     <div class="filter-group">
77         <label>Facultad:</label>
78         <select bind:value={selectedFacultad}>
79             <option value={null}>Todas las facultades</option>
80             {#each facultades as facultad}
81                 <option value={facultad.idFacultad}>
82                     {facultad.nombreFacultad}
83                 </option>
84             {/each}
85         </select>
86     </div>
87
88     <button on:click={clearFilters}>
89         Limpiar filtros
90     </button>
91 </div>
92
93 <div class="results">

```

```

94 <p>
95   Mostrando {filteredConvocatorias.length} resultados
96 </p>
97
98 {#each filteredConvocatorias as convocatoria}
99   <div class="result-item">
100     <h4>{convocatoria.solicitud?.materia?.nombreMateria}</h4>
101     <p>Estado: {convocatoria.estado}</p>
102   </div>
103 {/each}
104 </div>

```

Listing 10: Componente con filtros múltiples

La función `applyFilters` aplica los filtros de forma secuencial, reduciendo progresivamente el conjunto de resultados. Este enfoque permite combinar criterios de forma flexible y mantiene el código organizado.

3.1.3. Debouncing en búsquedas

Cuando la búsqueda implica consultas al servidor, es recomendable implementar debouncing para evitar realizar peticiones excesivas mientras el usuario escribe. El debouncing retrasa la ejecución de la búsqueda hasta que el usuario haya dejado de escribir durante un período determinado.

```

1 <!-- src/components/postulantes/PostulantesSearch.svelte -->
2 <script>
3   import { onMount, onDestroy } from 'svelte';
4
5   let searchTerm = '';
6   let searchResults = [];
7   let loading = false;
8   let debounceTimer;
9
10  $: {
11    clearTimeout(debounceTimer);
12    if (searchTerm.length >= 3) {
13      debounceTimer = setTimeout(() => {
14        performSearch(searchTerm);
15      }, 500); // Esperar 500ms despues de que el usuario deje de
16               escribir
17    } else {
18      searchResults = [];
19    }
20  }
21
22  async function performSearch(term) {
23    loading = true;
24
25    try {
26      const response = await fetch(
27        '/api/postulantes/buscar?q=${encodeURIComponent(term)}'

```

```

28     searchResults = await response.json();
29   } catch (error) {
30     console.error('Error en búsqueda:', error);
31     searchResults = [];
32   } finally {
33     loading = false;
34   }
35 }
36
37 onDestroy(() => {
38   clearTimeout(debounceTimer);
39 });
40 </script>
41
42 <div class="search-container">
43   <input
44     type="text"
45     bind:value={searchTerm}
46     placeholder="Buscar postulantes (min 3 caracteres)..."
47   />
48
49   {#if loading}
50     <p>Buscando...</p>
51   {:else if searchTerm.length >= 3}
52     <div class="results">
53       {#if searchResults.length === 0}
54         <p>No se encontraron resultados</p>
55       {:else}
56         {#each searchResults as postulante}
57           <div class="result-item">
58             <p>
59               {postulante.nombresPostulante}
60               {postulante.apellidosPostulante}
61             </p>
62             <p>{postulante.correoPostulante}</p>
63           </div>
64         {/each}
65       {/if}
66     </div>
67   {/if}
68 </div>

```

Listing 11: Implementación de debouncing

El debouncing reduce significativamente la cantidad de peticiones al servidor, mejorando el rendimiento y reduciendo la carga en el backend. El temporizador se limpia automáticamente cuando el componente se desmonta, previniendo fugas de memoria.

3.2. Consultas al servidor con parámetros

Para conjuntos de datos grandes, es más eficiente realizar las consultas y filtrados en el servidor. El frontend envía los parámetros de búsqueda como query parameters en la URL,

y el backend retorna únicamente los resultados que cumplen los criterios especificados.

3.2.1. Servicio de consultas parametrizadas

El servicio encapsula la construcción de URLs con parámetros de búsqueda, facilitando la realización de consultas complejas al backend.

```
1 // src/services/consultasService.js
2 import { api } from './api';
3
4 export const consultasService = {
5   async buscarConvocatorias(params) {
6     const queryParams = new URLSearchParams();
7
8     if (params.texto) {
9       queryParams.append('q', params.texto);
10    }
11    if (params.estado) {
12      queryParams.append('estado', params.estado);
13    }
14    if (params.facultadId) {
15      queryParams.append('facultadId', params.facultadId);
16    }
17    if (params.fechaDesde) {
18      queryParams.append('fechaDesde', params.fechaDesde);
19    }
20    if (params.fechaHasta) {
21      queryParams.append('fechaHasta', params.fechaHasta);
22    }
23
24    const endpoint = '/convocatorias/buscar?${queryParams.
25      toString()}';
26    return await api.get(endpoint);
27  },
28  async buscarPostulantes(params) {
29    const queryParams = new URLSearchParams();
30
31    if (params.nombre) {
32      queryParams.append('nombre', params.nombre);
33    }
34    if (params.identificacion) {
35      queryParams.append('identificacion', params.identificacion)
36      ;
37    }
38    if (params.estadoPostulacion) {
39      queryParams.append('estado', params.estadoPostulacion);
40    }
41
42    const endpoint = '/postulantes/buscar?${queryParams.toString
43      ()}';
44    return await api.get(endpoint);
```

```

43   },
44
45   async getPostulacionesPorEstado(estado) {
46     return await api.get('/postulaciones?estado=${estado}');
47   },
48
49   async getEvaluacionesPendientes() {
50     return await api.get('/evaluaciones?estado=pendiente');
51   }
52 };

```

Listing 12: Servicio de consultas con parámetros

El uso de `URLSearchParams` facilita la construcción de query strings correctamente formateadas, manejando automáticamente la codificación de caracteres especiales.

3.2.2. Componente de búsqueda avanzada

El componente de búsqueda avanzada permite a los usuarios especificar múltiples criterios de búsqueda que se envían al backend para su procesamiento.

```

1  <!-- src/components/busqueda/BusquedaAvanzada.svelte -->
2  <script>
3    import { consultasService } from
4      '../services/consultasService';
5
6    let searchParams = {
7      texto: '',
8      estado: '',
9      facultadId: null,
10     fechaDesde: '',
11     fechaHasta: ''
12   };
13
14   let resultados = [];
15   let loading = false;
16   let error = null;
17   let hasSearched = false;
18
19   async function handleSearch() {
20     loading = true;
21     error = null;
22     hasSearched = true;
23
24     try {
25       resultados = await consultasService.buscarConvocatorias(
26         searchParams
27       );
28     } catch (err) {
29       error = 'Error al realizar la busqueda';
30       resultados = [];
31     } finally {
32       loading = false;

```



```

33     }
34 }
35
36 function handleReset() {
37     searchParams = {
38         texto: '',
39         estado: '',
40         facultadId: null,
41         fechaDesde: '',
42         fechaHasta: ''
43     };
44     resultados = [];
45     hasSearched = false;
46     error = null;
47 }
48 </script>
49
50 <div class="search-panel">
51     <h2>Busqueda avanzada de convocatorias</h2>
52
53     <form on:submit|preventDefault={handleSearch}>
54         <div class="form-row">
55             <div class="form-group">
56                 <label>Texto de busqueda:</label>
57                 <input
58                     type="text"
59                     bind:value={searchParams.texto}
60                     placeholder="Materia, carrera o area"
61                 />
62             </div>
63
64             <div class="form-group">
65                 <label>Estado:</label>
66                 <select bind:value={searchParams.estado}>
67                     <option value="">Todos</option>
68                     <option value="abierta">Abierta</option>
69                     <option value="cerrada">Cerrada</option>
70                 </select>
71             </div>
72         </div>
73
74         <div class="form-row">
75             <div class="form-group">
76                 <label>Fecha desde:</label>
77                 <input
78                     type="date"
79                     bind:value={searchParams.fechaDesde}
80                 />
81             </div>
82
83             <div class="form-group">

```

```

84     <label>Fecha hasta:</label>
85     <input
86         type="date"
87         bind:value={searchParams.fechaHasta}
88     />
89 </div>
90 </div>
91
92 <div class="form-actions">
93     <button type="button" on:click={handleReset}>
94         Limpiar
95     </button>
96     <button type="submit" disabled={loading}>
97         {loading ? 'Buscando...' : 'Buscar'}
98     </button>
99 </div>
100 </form>
101
102 {#if loading}
103     <div class="loading">Buscando convocatorias...</div>
104 {:else if error}
105     <div class="error">{error}</div>
106 {:else if hasSearched}
107     <div class="results-section">
108         <h3>Resultados ({resultados.length})</h3>
109
110         {#if resultados.length === 0}
111             <p>No se encontraron convocatorias con los criterios
112                 especificados</p>
113         {:else}
114             <div class="results-list">
115                 {#each resultados as convocatoria}
116                     <div class="result-card">
117                         <h4>
118                             {convocatoria.solicitud?.materia?.nombreMateria}
119                         </h4>
120                         <p>
121                             Carrera:
122                             {convocatoria.solicitud?.materia?.carrera
123                                 ?.nombreCarrera}
124                         </p>
125                         <p>Estado: {convocatoria.estado}</p>
126                         <p>Cierre: {convocatoria.fechaCierre}</p>
127                     </div>
128                 {/each}
129             </div>
130         {:/if}
131     </div>
132 </div>

```

Listing 13: Componente de búsqueda avanzada

El componente mantiene un estado `hasSearched` para distinguir entre el estado inicial (sin búsqueda realizada) y el resultado de una búsqueda que no encontró resultados, proporcionando mensajes apropiados en cada caso.

3.3. Ordenamiento de resultados

El ordenamiento permite a los usuarios reorganizar los resultados según diferentes criterios, facilitando la localización de información relevante. El ordenamiento puede implementarse en el cliente o en el servidor, siguiendo la misma lógica que el filtrado.

3.3.1. Ordenamiento del lado del cliente

Para conjuntos de datos ya cargados en el cliente, el ordenamiento se implementa mediante la función `sort` de JavaScript, aprovechando la reactividad de Svelte para actualizar la vista automáticamente.

```

1 <!-- src/components/convocatorias/ConvocatoriasSorted.svelte -->
2 <script>
3   import { convocatoriasStore } from
4     '../stores/convocatoriasStore';
5
6   let sortField = 'fechaCierre';
7   let sortDirection = 'asc';
8
9   $: sortedConvocatorias = sortConvocatorias(
10     $convocatoriasStore.convocatorias,
11     sortField,
12     sortDirection
13   );
14
15   function sortConvocatorias(convocatorias, field, direction) {
16     const sorted = [...convocatorias].sort((a, b) => {
17       let valueA, valueB;
18
19       switch(field) {
20         case 'fechaCierre':
21           valueA = new Date(a.fechaCierre);
22           valueB = new Date(b.fechaCierre);
23           break;
24         case 'materia':
25           valueA = a.solicitud?.materia?.nombreMateria || '';
26           valueB = b.solicitud?.materia?.nombreMateria || '';
27           break;
28         case 'estado':
29           valueA = a.estado;
30           valueB = b.estado;
31           break;
32         default:
33           return 0;

```

```

34     }
35
36     if (valueA < valueB) {
37         return direction === 'asc' ? -1 : 1;
38     }
39     if (valueA > valueB) {
40         return direction === 'asc' ? 1 : -1;
41     }
42     return 0;
43 });
44
45 return sorted;
46 }
47
48 function handleSort(field) {
49     if (sortField === field) {
50         sortDirection = sortDirection === 'asc' ? 'desc' : 'asc';
51     } else {
52         sortField = field;
53         sortDirection = 'asc';
54     }
55 }
56 </script>
57
58 <div class="table-container">
59     <table>
60         <thead>
61             <tr>
62                 <th on:click={() => handleSort('materia')}>
63                     Materia
64                     {#if sortField === 'materia'}
65                         {sortDirection === 'asc' ? ' ' : ' '}
66                     {/if}
67                 </th>
68                 <th on:click={() => handleSort('estado')}>
69                     Estado
70                     {#if sortField === 'estado'}
71                         {sortDirection === 'asc' ? ' ' : ' '}
72                     {/if}
73                 </th>
74                 <th on:click={() => handleSort('fechaCierre')}>
75                     Fecha de cierre
76                     {#if sortField === 'fechaCierre'}
77                         {sortDirection === 'asc' ? ' ' : ' '}
78                     {/if}
79                 </th>
80             </tr>
81         </thead>
82         <tbody>
83             {#each sortedConvocatorias as convocatoria}
84                 <tr>

```

```

85         <td>
86             {convocatoria.solicitud?.materia?.nombreMateria}
87         </td>
88         <td>{convocatoria.estado}</td>
89         <td>{convocatoria.fechaCierre}</td>
90     </tr>
91     {/each}
92 </tbody>
93 </table>
94 </div>

```

Listing 14: Componente con ordenamiento

La función de ordenamiento crea una copia del array original antes de ordenarlo, evitando mutar el estado directamente. Los encabezados de columna son clickeables, permitiendo cambiar el campo y la dirección de ordenamiento mediante interacción del usuario.

3.3.2. Paginación de resultados

Para conjuntos de datos grandes, la paginación permite dividir los resultados en páginas más pequeñas, mejorando el rendimiento y la usabilidad de la interfaz.

```

1  <!-- src/components/shared/PaginatedList.svelte -->
2  <script>
3      export let items = [];
4      export let itemsPerPage = 10;
5
6      let currentPage = 1;
7
8      $: totalPages = Math.ceil(items.length / itemsPerPage);
9      $: paginatedItems = items.slice(
10         (currentPage - 1) * itemsPerPage,
11         currentPage * itemsPerPage
12     );
13
14     function goToPage(page) {
15         if (page >= 1 && page <= totalPages) {
16             currentPage = page;
17         }
18     }
19
20     function nextPage() {
21         goToPage(currentPage + 1);
22     }
23
24     function prevPage() {
25         goToPage(currentPage - 1);
26     }
27 </script>
28
29 <div class="paginated-container">
30     <slot name="items" items={paginatedItems} />
31

```

```
32 {#if totalPages > 1}
33   <div class="pagination">
34     <button
35       on:click={prevPage}
36       disabled={currentPage === 1}
37     >
38       Anterior
39     </button>
40
41     <span>
42       Pagina {currentPage} de {totalPages}
43     </span>
44
45     <button
46       on:click={nextPage}
47       disabled={currentPage === totalPages}
48     >
49       Siguiente
50     </button>
51
52     <select
53       bind:value={itemsPerPage}
54       on:change={() => currentPage = 1}
55     >
56       <option value={10}>10 por pagina</option>
57       <option value={25}>25 por pagina</option>
58       <option value={50}>50 por pagina</option>
59     </select>
60   </div>
61 {/if}
62 </div>
```

Listing 15: Componente con paginación

El componente de paginación es reutilizable y acepta cualquier lista de items mediante props, calculando automáticamente el número total de páginas y los elementos a mostrar en la página actual. El cambio en el número de elementos por página reinicia la paginación a la primera página para mantener la coherencia.

4. Tablas HTML dinámicas y visualización de datos

En el Sistema de Gestión de Titulación, el módulo de *Documentos Habilitantes* permite visualizar y dar seguimiento a los documentos requeridos durante el proceso de titulación (por ejemplo: cédula, certificado de votación, título, entre otros). El objetivo de este módulo es presentar la información de manera clara y reactiva mediante una tabla HTML dinámica, consumiendo datos reales desde un backend REST.

El flujo de integración considerado sigue el esquema BD → Backend → Frontend. En la base de datos (PostgreSQL) se registran los documentos habilitantes; el backend (Spring Boot) expone dichos registros mediante un endpoint REST; finalmente, el frontend (Svelte) consume el JSON y genera dinámicamente la tabla, incorporando estados de carga, error y ausencia de datos. :contentReference[oaicite:1]index=1

4.1. Endpoint REST para lectura de documentos habilitantes

En el backend se expone un endpoint REST encargado de devolver al frontend la lista de documentos habilitantes en formato JSON. La ruta general del servicio es:

- **Método:** GET
- **Ruta:** /documentos-habilitantes
- **Respuesta:** lista de documentos con campos como nombre, fecha de subida, estado y observaciones. :contentReference[oaicite:2]index=2

Este endpoint se implementa utilizando arquitectura por capas (Entity, Repository, Service y Controller), lo cual facilita el mantenimiento del sistema y permite evolucionar la lógica sin afectar directamente al frontend. :contentReference[oaicite:3]index=3

4.2. Consumo de datos reales desde Svelte

En Svelte, la función de ciclo de vida `onMount()` permite ejecutar lógica cuando el componente se renderiza por primera vez. Para el módulo de titulación, se utiliza `onMount()` para invocar el endpoint REST y cargar la lista de documentos habilitantes.

```
1 <script lang="ts">
2   import { onMount } from 'svelte';
3
4   type DocumentoHabilitante = {
5     nombre: string;
6     fechaSubida: string;
7     estado: 'APROBADO' | 'PENDIENTE' | 'RECHAZADO';
8     observaciones: string;
9   };
10
11   let documentos: DocumentoHabilitante[] = [];
12   let loading = true;
13   let error: string | null = null;
14
15   onMount(async () => {
16     try {
17       const response = await fetch('/documentos-habilitantes');
18       if (!response.ok) throw new Error('No se pudo obtener la
19         lista de documentos');
20       documentos = await response.json();
21     } catch (e) {
22       error = (e as Error).message;
23     } finally {
24       loading = false;
25     }
26   });
27 </script>
```

Listing 16: Consumo de documentos habilitantes con `onMount` y `fetch`

Este enfoque garantiza que el módulo funcione con datos reales registrados en el sistema y que la interfaz se mantenga alineada con la información del proceso académico. Además, permite incorporar fácilmente validaciones y controles posteriores según el rol del usuario (por ejemplo, estudiante o administrador del proceso de titulación). :contentReference[oaicite:4]index=4

4.3. Renderizado dinámico de filas con { #each }

Una tabla dinámica se construye generando filas en función del contenido recibido desde el backend. En Svelte, esto se implementa con el bloque { #each }, que recorre la colección y produce una fila por cada documento habilitante.

```

1  {#if loading}
2    <p>Cargando documentos...</p>
3  {:else if error}
4    <p class="error">Error: {error}</p>
5  {:else if documentos.length === 0}
6    <p>No existen documentos habilitantes registrados.</p>
7  {:else}
8    <table class="tabla">
9      <thead>
10         <tr>
11             <th>Documento</th>
12             <th>Fecha de subida</th>
13             <th>Estado</th>
14             <th>Observaciones</th>
15         </tr>
16     </thead>
17     <tbody>
18         {#each documentos as doc}
19             <tr>
20                 <td>{doc.nombre}</td>
21                 <td>{doc.fechaSubida}</td>
22                 <td>
23                     <span class="badge {doc.estado.toLowerCase()}>
24                         {doc.estado}
25                     </span>
26                 </td>
27                 <td>{doc.observaciones}</td>
28             </tr>
29         {/each}
30     </tbody>
31 </table>
32 {/if}

```

Listing 17: Renderizado dinámico de la tabla con { #each }

Con este patrón, la tabla se adapta automáticamente al volumen de información. Si se agregan nuevos documentos en el backend, la interfaz los reflejará sin necesidad de modificar el código del componente. Esto favorece la escalabilidad del módulo y reduce riesgos de inconsistencias entre capas. :contentReference[oaicite:5]index=5

4.4. Diseño institucional y legibilidad por estados

Dado que el módulo pertenece a un sistema universitario, resulta útil incorporar estilos que mejoren la lectura y permitan identificar rápidamente el estado de cada documento. Para ello se utilizan etiquetas (*badges*) con estilos diferenciados por estado: aprobado, pendiente y rechazado. :contentReference[oaicite:6]index=6

```

1 <style>
2   .tabla {
3     width: 100%;
4     border-collapse: collapse;
5     background: #fff;
6     border-radius: 8px;
7     overflow: hidden;
8   }
9
10  .tabla th, .tabla td {
11    padding: 10px 12px;
12    border-bottom: 1px solid #eaeaea;
13    text-align: left;
14  }
15
16  .badge {
17    padding: 4px 10px;
18    border-radius: 999px;
19    font-size: 0.85rem;
20    font-weight: 600;
21  }
22
23  .badge.aprobado { background: #d6f5d6; }
24  .badge.pendiente { background: #fff3cd; }
25  .badge.rechazado { background: #f8d7da; }
26
27  .error { font-weight: 600; }
28 </style>

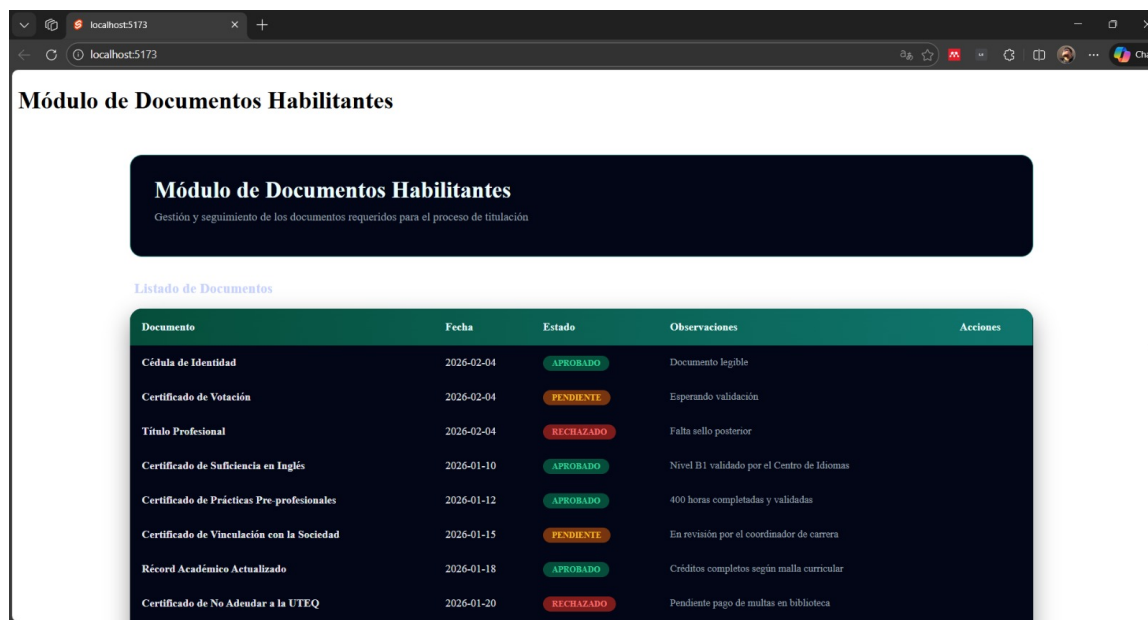
```

Listing 18: Estilos para tabla institucional y badges de estado

Este diseño mejora la experiencia de usuario al reducir el esfuerzo de interpretación del estado, lo cual resulta importante cuando se revisan varios documentos dentro del proceso de titulación.

4.5. Evidencia visual del módulo

Es válido incluir una captura del frontend, ya que funciona como evidencia del comportamiento real del componente y complementa los listings anteriores. Se recomienda que la imagen muestre: la tabla renderizada, al menos tres documentos con estados diferentes y la estructura tipo panel institucional.



Documento	Fecha	Estado	Observaciones	Acciones
Cédula de Identidad	2026-02-04	APROBADO	Documento legible	
Certificado de Votación	2026-02-04	PENDIENTE	Esperando validación	
Título Profesional	2026-02-04	RECHAZADO	Falta sello posterior	
Certificado de Suficiencia en Inglés	2026-01-10	APROBADO	Nivel B1 validado por el Centro de Idiomas	
Certificado de Prácticas Pre-profesionales	2026-01-12	APROBADO	400 horas completadas y validadas	
Certificado de Vinculación con la Sociedad	2026-01-15	PENDIENTE	En revisión por el coordinador de carrera	
Récord Académico Actualizado	2026-01-18	APROBADO	Créditos completos según malla curricular	
Certificado de No Adeudar a la UTEQ	2026-01-20	RECHAZADO	Pendiente pago de multas en biblioteca	

Figura 4: Módulo de Documentos Habilitantes del Sistema de Gestión de Titulación. La tabla se renderiza dinámicamente desde datos obtenidos del endpoint REST e identifica estados mediante etiquetas visuales.

5. Generación de reportes e impresión en PDF

El Sistema de Titulaciones Universitarias incluye un módulo de reportes que permite visualizar información consolidada de facultades y carreras, así como generar documentos imprimibles en formato PDF. Este módulo fue desarrollado utilizando Svelte en el frontend, consumiendo servicios REST expuestos por el backend Spring Boot y utilizando PostgreSQL como sistema gestor de base de datos.

El objetivo principal del módulo es permitir al usuario consultar información estructurada y generar un documento formal que pueda imprimirse o guardarse como archivo PDF, utilizando las capacidades nativas del navegador y estilos CSS especializados para impresión. :contentReference[oaicite:1]index=1

5.1. Componente principal del reporte

El componente `ReporteFacultades.svelte` implementa la lógica completa del reporte. Este componente recibe como parámetro la URL base del backend mediante una propiedad (`prop`), lo que permite desacoplar la configuración del servidor de la lógica del reporte.

```

1 <script>
2   export let API;
3
4   let datos    = [];
5   let loading  = false;
6   let error    = '';
7   let buscado  = false;
8
9   const hoy = new Date().toLocaleDateString('es-EC', {
10     day: '2-digit', month: 'long', year: 'numeric'
11   });
12 </script>
```

Listing 19: Recepción de la URL del backend como prop

Las variables cumplen los siguientes propósitos:

- `datos`: almacena las facultades obtenidas desde el backend.
- `loading`: indica si la aplicación está esperando una respuesta del servidor.
- `error`: contiene el mensaje de error en caso de fallo.
- `buscado`: indica si ya se realizó una consulta.
- `hoy`: almacena la fecha actual para mostrarla en el reporte.

5.2. Consulta de datos al backend

La función `buscar()` realiza la solicitud HTTP al backend utilizando la función nativa `fetch`. Esta función consulta el endpoint REST que devuelve la información necesaria para generar el reporte.

```

1 async function buscar() {
2
3   loading = true;
4   error   = '';
5   buscado = false;
6
7   try {
8
9     const r = await fetch(`${API}/reportes/facultades`);
10
11    if (!r.ok)
12      throw new Error('Error del servidor: ${r.status}');
13
14    datos    = await r.json();
15    buscado = true;
16
17  } catch (e) {
18
19    error = e.message;
20
21  } finally {
22
23    loading = false;
24
25  }
26 }
```

Listing 20: Función para obtener datos del reporte desde el backend

Esta función permite integrar el frontend con el backend Spring Boot, el cual obtiene los datos desde PostgreSQL y los devuelve en formato JSON estructurado.

5.3. Cálculo reactivo de información

Svelte permite definir variables reactivas mediante el operador `$:`, el cual recalcula automáticamente su valor cuando cambian los datos dependientes.

```

1 $: totalCarreras = datos.reduce((suma, facultad) => {
2   return suma + (facultad.totalCarreras || 0);
3 }, 0);
```

Listing 21: Cálculo reactivo del total de carreras

Esta variable se actualiza automáticamente cuando cambia el contenido del arreglo `datos`, eliminando la necesidad de invocar funciones manualmente para recalcular valores.

5.4. Renderizado del reporte en pantalla

La información del reporte se renderiza dinámicamente utilizando el bloque { #each }, que permite iterar sobre los elementos del arreglo y generar la interfaz de forma automática.

```

1 {#if buscado}
2
3   <div class="resumen">
4     <div>
5       <span>{datos.length}</span>
6       <span>Facultades</span>
7     </div>
8
9     <div>
10      <span>{totalCarreras}</span>
11      <span>Carreras en total</span>
12    </div>
13  </div>
14
15  {#each datos as f, fi}
16
17    <div class="fac-card">
18
19      <div class="fac-head">
20        <span>{fi + 1}</span>
21        <div>{f.nombre}</div>
22        <div>{f.universidad}</div>
23      </div>
24
25      <table>
26        <thead>
27          <tr>
28            <th>#</th>
29            <th>Carrera</th>
30          </tr>
31        </thead>
32
33        <tbody>
34          {#each f.carreras as c, ci}
35            <tr>
36              <td>{ci + 1}</td>
37              <td>{c.nombre}</td>
38            </tr>
39          {/each}
40        </tbody>
41
42      </table>
43
44    </div>
45
46  {/each}

```

```
47  
48 {/if}
```

Listing 22: Renderizado dinámico del reporte

Este diseño permite visualizar de forma estructurada las facultades y sus respectivas carreras.

5.5. Generación del PDF mediante el navegador

El reporte puede convertirse en PDF utilizando la función `window.print()`, que abre el diálogo de impresión del navegador.

```
1 function imprimir() {  
2     window.print();  
3 }
```

Listing 23: Función para generar el PDF

Esta función utiliza estilos CSS especializados que ocultan elementos innecesarios y muestran únicamente el contenido del reporte durante la impresión.

5.6. Estilos CSS para impresión

El comportamiento de impresión se controla mediante reglas CSS utilizando `@media print`.

```
1 @media screen {  
2     .print-only {  
3         display: none;  
4     }  
5 }  
6  
7 @media print {  
8  
9     .no-print {  
10        display: none !important;  
11    }  
12  
13    @page {  
14        size: A4 portrait;  
15        margin: 2cm 1.5cm;  
16    }  
17  
18 }
```

Listing 24: Reglas CSS para impresión del reporte

Estas reglas permiten mostrar únicamente el reporte durante la impresión y ocultar elementos interactivos del sistema.

5.7. Evidencia visual del módulo de reportes

La Figura 5 muestra el módulo de reportes en ejecución dentro del Sistema de Titulaciones Universitarias. Se observa la estructura del reporte, incluyendo facultades, carreras y el formato visual utilizado antes de generar el PDF.

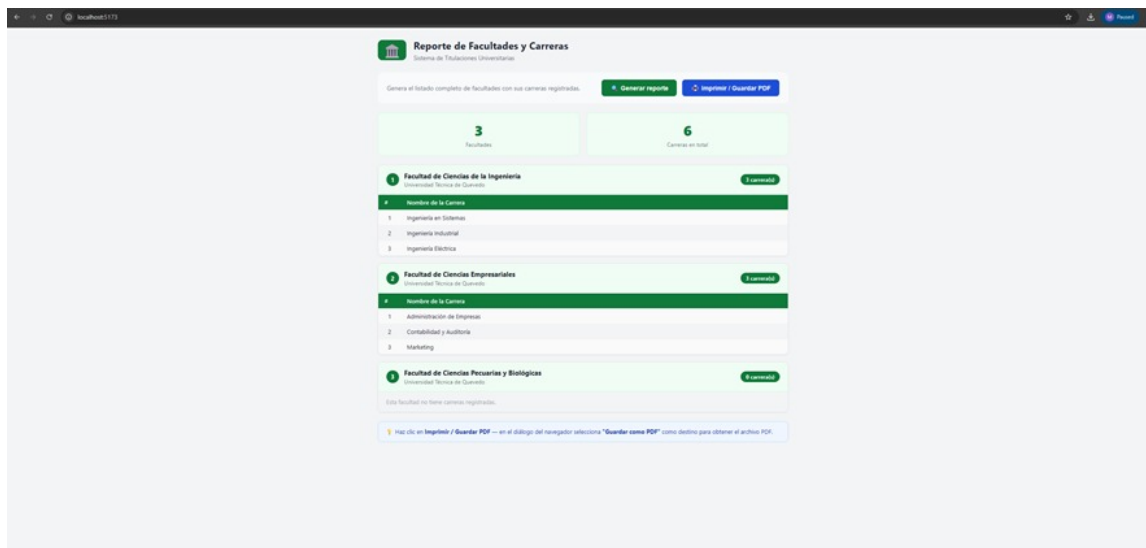


Figura 5: Interfaz del módulo de reportes del Sistema de Titulaciones Universitarias. El usuario puede visualizar la información y generar el documento en formato PDF.

6. Conclusiones del manual técnico

El presente manual técnico ha descrito la implementación práctica de un sistema frontend completo desarrollado con Svelte para aplicaciones web que consumen servicios REST. A través de las secciones presentadas, se han cubierto los aspectos fundamentales del desarrollo de interfaces modernas, incluyendo operaciones CRUD, consultas dinámicas, visualización de datos mediante tablas interactivas y generación de reportes.

La implementación de operaciones CRUD en Svelte demuestra cómo el framework facilita la gestión de estados asíncronos y la actualización reactiva de la interfaz. El sistema de reactividad integrado permite que los cambios en los datos se reflejen automáticamente en la vista, reduciendo la complejidad del código y mejorando la mantenibilidad del sistema. La separación entre servicios, stores y componentes proporciona una arquitectura clara que facilita la evolución y el mantenimiento del código.

El sistema de consultas y filtros implementado ilustra la flexibilidad que ofrece Svelte para construir interfaces interactivas que responden inmediatamente a las acciones del usuario. Las técnicas de debouncing y optimización de peticiones al servidor contribuyen a un mejor rendimiento y experiencia de usuario, especialmente en escenarios con grandes volúmenes de datos.

Las tablas dinámicas desarrolladas muestran cómo componentes reutilizables pueden proporcionar funcionalidades avanzadas como ordenamiento, selección múltiple, expansión de filas y carga progresiva. La arquitectura basada en componentes permite que estas funcionalidades se integren de forma modular en diferentes partes del sistema sin duplicar código.

Finalmente, la implementación de reportes demuestra las diferentes estrategias disponibles para presentar información consolidada, desde visualizaciones en pantalla hasta exportación de datos y generación de documentos en el servidor. La elección entre estas estrategias depende de los requisitos específicos de cada caso de uso, considerando factores como complejidad del formato, volumen de datos y necesidades de los usuarios.

En conjunto, el manual proporciona una guía práctica para desarrollar aplicaciones frontend profesionales con Svelte, integrando buenas prácticas de desarrollo web moderno y aprovechando las capacidades del framework para construir interfaces reactivas, eficientes y mantenibles que consumen servicios REST de forma segura y robusta.